

《高级语言程序设计》考试在12月21日，
9：30-11：00，铜盘A608。

《高级语言程序设计实践》考试在12月
28日9：30-11：30，铜盘A205。

A powerful index.

指针

理论课程



福州大学
FUZHOU UNIVERSITY

内容要点

- 指针

- 指针的声明、赋值与使用
- 指针的操作、指针和数组之间的关系
- 指针的应用场景

目录

	3	指针
	3.1	指针的声明和赋值
	3.2	指针的操作
	3.3	指针和数组
	3.4	指针的应用

指针

- 物理意义：指向内存地址（address）
- 内存空间占用：4或8字节，由应用程序的架构决定
- 取值范围：无符号整数
 - 理论上，最小值：0；最大值： $2^{32}-1$ 或 $2^{64}-1$ （程序架构）
 - 实际上，为当前程序可以访问的地址范围
- 指针的属性
 - 指针指向的数据类型
 - 指针的值
 - 指针所在的地址

```
int *p = NULL;
```

这里声明一个指针。
指向数据类型为整型。
指针的值为0。（初始化）
指针的地址为0xffffcc18。（每次运行不同）

指针修饰符

* 修饰符

功能	标记声明的变量为指针		
格式	type *var;		
参数	type	类型名，指针指向的数据类型	
	var	变量名或变量列表，待声明的指针变量	
示例	int *p;		声明指向整型的指针变量p。
	int *p = NULL;		声明整型指针变量p赋值为NULL。
	int *p, *q, r;		声明整型指针变量p和q，和整型变量r。
	int* p, q, r;		*是修饰p而非int，不推荐和int连写。
说明	1. 列表形式声明语句，每个指针前应 单独 书写修饰符。 2. 声明不应该把int*连着写，以免读者误认p,q为int*类型。		

指针的声明

- 格式 `<类型> *<指针名>;`

- 声明语句中使用**修饰符** * 标记一个变量为指针
- 列表形式声明语句，每个指针前应**单独**书写修饰符。
- 例如：

```
int *p, *q, r; // r is not a pointer
```

- 声明时赋初始值

- 指针应先声明，再赋值，最后使用
- 通常以空指针NULL表示无意义

取址操作符

& 操作符

功能	求地址	
格式	<i>&function</i> 或 <i>&lvalue</i> 或 <i>&refer</i>	
参数	<i>function</i>	函数名
	<i>lvalue</i>	左值，位域和 <code>register</code> 存储类除外
	<i>refer</i>	间接引用，为指针的解引用或数组下标运算
示例	<code>&printf</code>	求printf函数或变量x的地址
	<code>&*p</code>	求p指向内存的地址，即p的值但不是左值
	<code>&a[3]</code>	求数组a第4个元素的地址，即a+3，允许越界
说明	1. 两次运行的结果不一定相同；不同程序架构和编译器下，一些结果的规律可能不同。	

```
#include <stdio.h>
```

```
int main() {
```

```
    int x = 10;
```

```
    int *p = &x;
```

```
    printf("%p\n", p);
```

```
    printf("%p\n", &p);
```

```
    printf("%p\n", &*p);
```

```
    return 0;
```

```
}
```

```
00000000000062FE4C
```

```
00000000000062FE40
```

```
00000000000062FE4C
```

```

/* loccheck.c -- checks to see where variables are stored */
#include <stdio.h>
void mikado(int);           /* declare function */
int main(void) {
    int pooh = 2, bah = 5;   /* local to main() */
    printf("In main(), pooh = %d and &pooh = %p\n", pooh, &pooh);
    printf("In main(), bah = %d and &bah = %p\n", bah, &bah);
    mikado(pooh)
    return 0;
}

void mikado(int bah) {      /* define function */
    int pooh = 10;          /* local to mikado() */
    printf("In mikado(), pooh = %d and &pooh = %p\n", pooh,
&pooh);
    printf("In mikado(), bah = %d and &bah = %p\n", bah, &bah);
}

```

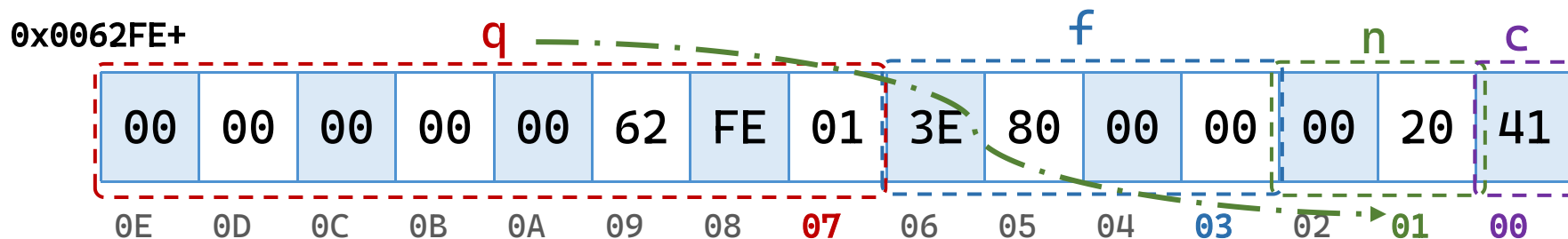
```

In main(), pooh = 2 and &pooh = 00C4F7B0
In main(), bah = 5 and &bah = 00C4F7A4
In mikado(), pooh = 10 and &pooh = 00C4F6C0
In mikado(), bah = 2 and &bah = 00C4F6D0

```

声明指针变量时的内存分配

- 指针也是数据类型，指针变量也有所在地址和值



变量名	内存地址	值	长度
c	0x0062FE00	'A'	1B
n	0x0062FE01	32	2B
f	0x0062FE03	0.25	4B
q	0x0062FE07	0x0062FE01	8B

```
char c = 'A';  
short n = 32;  
float f = 0.25;  
short *q = &n;
```

指针的赋值

- 指针应先声明后使用，先写后读
 - 如果程序涉及指针值的操作，应先赋值，再使用
 - 如果程序涉及间接寻址，还要求该值可间接寻址
 - 如果程序对间接寻址的值读取，还要求该地址空间先赋值
 - 通常以空指针 NULL（即：(void *)0）表示指针无定义

```
int *p;      // 声明，但此时 p 里是随机垃圾值
*p = 10;     // 错误，p 还没指向有效地址
```

```
int x;
int *p = &x; // p 指向 x 的地址
*p = 10;     // 合法，给 x 赋值
```

指针的赋值

- 指针应先声明后使用，先写后读
 - 如果程序涉及指针值的操作，应先赋值，再使用
 - 如果程序涉及间接寻址，还要求该值可间接寻址
 - 如果程序对间接寻址的值读取，还要求该地址空间先赋值
 - 通常以空指针 NULL（即：(void *)0）表示指针无定义

```
int *p = NULL;
```

```
*p = 10; // NULL 地址不可间接访问，程序会崩溃
```

间接寻址就是解引用指针（用 *p），这会去访问指针所指向的内存地址。

指针的赋值

- 指针应先声明后使用，先写后读
 - 如果程序涉及指针值的操作，应先赋值，再使用
 - 如果程序涉及间接寻址，还要求该值可间接寻址
 - 如果程序对间接寻址的值读取，还要求该地址空间先赋值
 - 通常以空指针 NULL（即：(void *)0）表示指针无定义

```
int x;    // 未初始化
int *p = &x;
printf("%d\n", *p); // 逻辑上错误，x 里是未定义值
```

```
int x = 0;
int *p = &x;
printf("%d\n", *p); // 已赋值，可安全读取
```

指针的赋值

- 指针应先声明后使用，先写后读
 - 如果程序涉及指针值的操作，应先赋值，再使用
 - 如果程序涉及间接寻址，还要求该值可间接寻址
 - 如果程序对间接寻址的值读取，还要求该地址空间先赋值
- 指针的赋值

赋值类型	示例
空指针	<code>int *p = NULL;</code>
同类型的变量取地址	<code>int q = 0, *p = &q;</code>
同类型的指针构成的表达式	<code>int *p = &q, *r = p + 1;</code>

与常量相关指针的声明

- 声明时标记`const`，离其最近的修饰符`*`为只读对象。

示例	<code>p++</code>	<code>(*p)++</code>	<code>(**p)++</code>
<code>int * const p = &q;</code>	×	✓	/
<code>const int * p = &q;</code>	✓	×	/
<code>int pq = &q; int * const * p = &pq;</code>	✓	×	✓
<code>int * const * const p = &pq;</code>	×	×	✓
<code>const int * * const p = &pq;</code>	×	✓	×
<code>const int * const * const p = &pq;</code>	×	×	×

- 设置`const`后对只读对象赋值，将提示编译错误

[Error] increment of read-only variable 'p'

[Error] increment of read-only location '*(const int * const*)p'

[Error] increment of read-only location '*(const int *)*(const int * const*)p'

目录

	3	指针
	3.1	指针的声明和赋值
	3.2	指针的操作
	3.3	指针和数组
	3.4	指针的应用

解引用操作符

左值 (lvalue)

指的是表示内存中某个可识别位置的表达式，可以出现在赋值运算符 = 的左边。

* 操作符

功能	解引用，将指针值转换为一个左值。	
格式	<i>*pointer</i>	
参数	<i>pointer</i>	指针类型的表达式
返回值	成功	当 <i>pointer</i> 是函数指针时，返回该函数的代指符。 当 <i>pointer</i> 是对象指针时，返回对象的左值表达式。
	失败	解引用空指针、指向在生存期外的对象的指针（悬垂指针）、错误对齐的指针或拥有不确定值的指针是未定义行为。
示例	<i>*p</i>	求 <i>p</i> 指向的对象
说明	1. 无。	

指针的修饰符与操作符的区别

项目	修饰符	操作符
使用位置	声明语句中	表达式中
* 号	标记一个变量为指针	对指针解引用
[] 号	标记一个数组，指定长度	通过指定数组下标求元素
() 号	标记一个函数，指定参量	提升优先级，或函数调用，指定参数

```
int q = 1, *p = &q, r = *p;  
int x = 1, a[10] = {0}, r = a[1];  
int max(int x, int y) {  
    return (x>y)?x:y;  
}
```

这个例子里，蓝色的符号是修饰符，主要是声明语句中使用，而绿色的符号是操作符，主要是表达式中使用，尤其包括声明语句中用于初始化的表达式。

运算符的优先级

• 下表为与本节相关的优先级顺序

序号	符号	说明	序号	符号	说明
1 →	后缀++ --	后缀增减量	3	* / %	算术运算：乘除模
	()	函数调用	4	+ -	算术运算：加减
	[]	数组下标	6	< > <= >=	关系运算符：大小
	(type){list}	复合文字	7	== !=	关系运算符：相等
2 ←	前缀++ --	前缀增减量	11	&&	逻辑运算符：与
	+ -	正负号	12		逻辑运算符：或
	!	逻辑运算符：非	13	?:	三元条件运算符
	(type)	强制类型转换	14 ←	=	赋值
	*	间接寻址		+= -= *=	自增自减自乘自除
	&	取址		/= %=	自模
	sizeof	存储空间	15	,	逗号表达式

间接寻址和取址操作符

• 例题

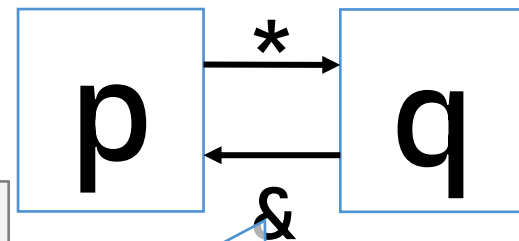
– 执行 `int q = 1, *p = &q;` 后, `*p` 的值为多少?

```
#include <stdio.h>
```

```
int main() {  
    int q = 1, * p = &q;  
    printf("&p=%p\n", &p);  
    printf(" p=%p\n", p);  
    printf("*p=%d\n", *p);  
    printf("&q=%p\n", &q);  
    printf(" q=%d\n", q);  
    return 0;  
}
```

这里*是修饰符, 与表达式中的*不同, 因此运行后p的值为&q的值。
不能认为*p的值为&q的值。

```
&p=0xffffcbcb8  
p=0xffffcbcb4  
*p=1  
&q=0xffffcbcb4  
q=1
```



*p即q, 即q所在空间;
p即&q, 即q所在地址;
&p即p所在地址。

指针指向的数据类型

- 指向类型不相同不能直接赋值
 - 元素类型不同的数组，含高维数组，参量列表不同的函数
- 无类型指针可以赋值给其他指针，再计算

指向数据类型	示例
无类型	<code>void *p;</code>
基本数据类型	<code>char q = 'A'; char *p = &q;</code>
数组	<code>int a[5] = {0}; int *p = a;</code>
指针	<code>int a = 0, *p = &a; int **pp = &p;</code>
结构体	<code>struct student *pstu;</code>
函数	<code>double fabs(double _X); double (*q)(double) = fabs;</code>

指针的操作

- 指针（含：数组名）可以执行以下操作
 - 指针加减不同于整数的加减，步长为所指向类型的大小
 - 数组名是常量，不能作为左值

操作	数组名	指针变量	示例
被赋值	×	✓	<code>ptr=&var</code>
求值或取值	✓	✓	<code>*ptr</code>
取指针地址	✓	✓	<code>&var</code>
加上或减去一个整数	✓	✓	<code>ptr+num</code>
自增或自减	×	✓	<code>ptr++</code>
求差值	✓	✓	<code>ptr1-ptr2</code>
比较大小	✓	✓	<code>ptr1<=ptr2</code>


```
// ptr_ops.c -- pointer operations
#include <stdio.h>
int main(void) {
    int urn[5] = {100,200,300,400,500};
    int * ptr1, * ptr2, * ptr3;
    ptr1 = urn;           // assign an address to a pointer
    ptr2 = &urn[2];       // ditto
    // dereference a pointer and take
    // the address of a pointer
    printf("pointer value, dereferenced pointer, pointer
address:\n");
    printf("ptr1 = %p, *ptr1 = %d, &ptr1 = %p\n", ptr1,
*ptr1, &ptr1);
    // pointer addition
    ptr3 = ptr1 + 4;
    printf("\nadding an int to a pointer:\n");
    printf("ptr1 + 4 = %p, *(ptr1 + 3) = %d\n", ptr1 + 4,
*(ptr1 + 3));
}
```

```
ptr1++;           // increment a pointer
printf("\nvalues after ptr1++:\n");
printf("ptr1 = %p, *ptr1 = %d, &ptr1 = %p\n", ptr1,
*ptr1, &ptr1);
ptr2--;           // decrement a pointer
printf("\nvalues after ptr2--:\n");
printf("ptr2 = %p, *ptr2 = %d, &ptr2 = %p\n",
ptr2, *ptr2, &ptr2);
--ptr1;           // restore to original value
++ptr2;           // restore to original value
printf("\nPointers reset to original values:\n");
printf("ptr1 = %p, ptr2 = %p\n", ptr1, ptr2);
// subtract one pointer from another
printf("\nsubtracting one pointer from another:\n");
printf("ptr2 = %p, ptr1 = %p, ptr2 - ptr1 = %td\n",
ptr2, ptr1, ptr2 - ptr1);
// subtract an integer from a pointer
printf("\nsubtracting an int from a pointer:\n");
```

```
printf("ptr3 = %p, ptr3 - 2 = %p\n",  
      ptr3, ptr3 - 2);
```

```
return 0;
```

```
}
```

pointer value, dereferenced pointer, pointer address:

ptr1 = 0xbf804b6c, *ptr1 = 100, &ptr1 = 0xbf804b60

adding an int to a pointer:

ptr1 + 4 = 0xbf804b7c, *(ptr1 + 3) = 400

values after ptr1++:

ptr1 = 0xbf804b70, *ptr1 = 200, &ptr1 = 0xbf804b60

values after --ptr2:

ptr2 = 0xbf804b70, *ptr2 = 200, &ptr2 = 0xbf804b64

Pointers reset to original values:

ptr1 = 0xbf804b6c, ptr2 = 0xbf804b74

subtracting one pointer from another:

ptr2 = 0xbf804b74, ptr1 = 0xbf804b6c, ptr2 - ptr1 = 2

subtracting an int from a pointer:

ptr3 = 0xbf804b7c, ptr3 - 2 = 0xbf804b74

无类型指针

- 无类型指针 (`void *`)

- 目标类型未知的情况

- 常用于函数参量，表示任何目标类型的指针都兼容
 - 调用函数时需清楚原始传入的类型，并及时将参量转为对应类型（比如 `float*`、`int*`）。

```
void qsort(void* base, size_t nmem, size_t size,  
          int(*compar)(const void*, const void*));  
int compare(const void* v1, const void* v2) {  
    return *(float*)v1 - *(float*)v2;  
}
```

- 可直接赋值给任何类型指针或被赋值不需强制类型转换
 - 不可以间接寻址或算术运算（目标空间大小未知）

指针的类型兼容性

- 不同类型指针的相互转换

- 无强制类型转换将提示告警

```
warning: assignment to 'float *' from incompatible pointer type 'int *'
[-Wincompatible-pointer-types]
```

- 不同类型指针相互转换可能无意义，且和端序有关

```
int n = 1819043144;
float x;
int * pi = &n;
float * pf = &x;
x = n;
pf = pi;
```

二进制	01001000	01100101	01101100	01101100
n	1819043144 $2^{30}+2^{27}+2^{22}+2^{21}+2^{18}+2^{16}+2^{14}+2^{13}+2^{11}$ $+2^{10}+2^6+2^5+2^3+2^2$			
x	1819043144			
*pi	1819043144			
pf	1143139122437582505939828736 $(1+2^{-2}+2^{-3}+2^{-6}+2^{-8}+2^{-10}+2^{-11}+2^{-13}+2^{-14}+2^{-18}+2^{-19}$ $+2^{-21}+2^{-22})\text{power}(2, 2^6+2^5+2^3+2^2+2^1)$			

数据类型与内存存储

- 同一段内存使用不同类型解析时的结果

HEX	48	65	6c	6c	6f	21	00	a3
CHAR	'H'	'e'	'l'	'l'	'o'	'!'	'\0'	'\xA3'
UCHAR	72	101	108	108	111	33	0	163
INT16	25928		27756		8559		−23808	
UINT16	25928		27756		8559		41728	
INT32	1819043144				−1560272529			
UINT32	1819043144				2734694767			
INT64	−6701319483083168440							
UINT64	11745424590626383176							
FLOAT	1.14314e+027				−6.94597e−018			
DOUBLE	−4.23295e−140							

指针的类型兼容性

- 不要将指向常量的指针赋值给指向变量的指针
 - 编译选项为-O0（无优化）时存在修改常量值的风险。
 - 在C中为警告，C++为编译错误

`const int *pn = &n;`

```
#include <stdio.h>
int main(void) {
    const int n = 13;
    int *p;
    const int **pp = &p;
    *pp = &n;
    *p = 16;
    printf("n = %d\n", n);
}
```

[Warning] initialization of 'const int **' from incompatible pointer type 'int **' [-Wincompatible-pointer-types]

```
#include <stdio.h>
int main(void) {
    const int n = 13;
    int *pn = &n;
    *pn = 10;
    printf("n = %d\n", n);
    return 0;
}
```

[Warning] initialization discards 'const' qualifier from pointer target type [-Wdiscarded-qualifiers]

目录

	3	指针
	3.2	指针的操作
	3.3	指针和数组
	3.4	指针的应用
	3.5	常见的编程错误

指针和数组的区别联系

- 数组是常量，指针是变量
 - 数组名称的值是该数组元素的首地址，不能被赋值
 - 指针可以用数组赋值
- 指针组成表达式的方法与数组相同
 - 指针间接寻址 $*(a+i)$ 与数组取元素 $a[i]$ 相同
 - 指针间接寻址 $((*a)+j)$ 与数组取元素 $a[i][j]$ 相同

```

/* order.c -- precedence in pointer operations */
#include <stdio.h>
int data[2] = {100, 200};
int moredata[2] = {300, 400};
int main(void)
{
    int * p1, * p2, * p3;
    p1 = p2 = data;
    p3 = moredata;
    printf("    *p1 = %d,    *p2 = %d,    *p3 = %d\n",
           *p1,    *p2,    *p3);
    printf("*p1++ = %d, *++p2 = %d, (*p3)++ = %d\n",
           *p1++,    *++p2,    (*p3)++);
    printf("    *p1 = %d,    *p2 = %d,    *p3 = %d\n",
           *p1,    *p2,    *p3);
    return 0;
}

```

```

    *p1 = 100,    *p2 = 100,    *p3 = 300
    *p1++ = 100, *++p2 = 200, (*p3)++ = 300
    *p1 = 200,    *p2 = 200,    *p3 = 301

```

```
// pnt_add.c -- pointer addition
```

```
#include <stdio.h>
```

```
#define SIZE 4
```

```
int main(void)
{
```

```
    short dates [SIZE];
```

```
    short * pti;
```

```
    short index;
```

```
    double bills[SIZE];
```

```
    double * ptf;
```

```
    pti = dates;    // assign address of array to pointer
```

```
    ptf = bills;
```

```
    printf("%23s %15s\n", "short", "double");
```

```
    for (index = 0; index < SIZE; index ++)
```

```
        printf("pointers + %d: %10p %10p\n",
               index, pti + index, ptf + index);
```

```
    return 0;
```

```
}
```

	short	double
pointers + 0:	0046F9D0	0046F990
pointers + 1:	0046F9D2	0046F998
pointers + 2:	0046F9D4	0046F9A0
pointers + 3:	0046F9D6	0046F9A8

```

/* day_mon3.c -- uses pointer notation */
#include <stdio.h>
#define MONTHS 12

int main(void)
{
    int days[MONTHS] = {31,28,31,30,31,30,31,31,30,31,30,31};
    int index;

    for (index = 0; index < MONTHS; index++)
        printf("Month %2d has %d days.\n", index + 1,
               *(days + index));    // same as days[index]

    return 0;
}

```

Month 1 has 31 days.
 Month 2 has 28 days.
 Month 3 has 31 days.
 (此处省略数行)
 Month 12 has 31 days.

指向数组的指针作为函数参量

- 指针作为函数的参量

```
int sum(int ar[], int n)
int sum(int * ar) // 也要加 int n
```

 - 两种形式是等价的，ar实际上都是指针
 - 当ar表示数组时，应使用数组形式；当ar表示指针时，应使用指针形式，以免读者误解。
- 函数参量含有数组时，应另有参量指示数组长度
 - 只有在作用域内可通过变量名求出长度
 - 不论何种形式的参量，在函数中都是指针，容易导致越界

在函数外、数组自己定义的地方：
sizeof(a) / sizeof(a[0])

在函数里面都只是 int *，这时候：
sizeof(a) 得到的是指针本身的大小（4/8 字节）

```
// sum_arr1.c -- sums the elements of an array
// use %u or %lu if %zd doesn't work
#include <stdio.h>
#define SIZE 10
int sum(int ar[], int n);
int main(void) {
    int marbles[SIZE] = { 20,10,5,39,4,16,19,26,31,20 };
    long answer;
    answer = sum(marbles, SIZE);
    printf("The total number of marbles is %ld.\n", answer);
    printf("The size of marbles is %zd bytes.\n", sizeof marbles);
    return 0;
}

int sum(int ar[], int n) {           // how big an array?
    int i;
    int total = 0;
    for (i = 0; i < n; i++)
        total += ar[i];
    printf("The size of ar is %zd bytes.\n", sizeof ar);
    return total;
}
```

The size of ar is 4 bytes.
 The total number of marbles is 190.
 The size of marbles is 40 bytes.

在参量中标记只读

- 可以分别对数组元素和指针变量设置只读

只读对象	标记位置	示例代码
数组元素	类型之前	<code>void show_array(const double ar[], int n);</code>
指针变量	类型之后	<code>void show_array(double ar[const], int n);</code> <code>void show_array(double * const ar, int n);</code>
二者不改	二者兼顾	<code>void show(const double ar[const], int n);</code> <code>void show(const double * const ar, int n);</code>

- 设置`const`后对只读对象赋值，将提示编译错误

```
error: assignment of read-only location '*(ar + 16)'  
error: assignment of read-only parameter 'ar'
```

```
/* arf.c -- array functions */
#include <stdio.h>
#define SIZE 5
void show_array(const double ar[], int n);
void mult_array(double ar[], int n, double mult);

int main(void)
{
    double dip[SIZE] = {20.0, 17.66, 8.2, 15.3, 22.22};

    printf("The original dip array:\n");
    show_array(dip, SIZE);
    mult_array(dip, SIZE, 2.5);
    printf("The dip array after calling mult_array():\n");
    show_array(dip, SIZE);

    return 0;
}
```



```

/* displays array contents */
void show_array(const double ar[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%8.3f ", ar[i]);
    putchar('\n');
}

/* multiplies each array member by the same multiplier */
void mult_array(double ar[], int n, double mult)
{
    int i;
    for (i = 0; i < n; i++)
        ar[i] *= mult;
}

```

The original dip array:

20.000	17.660	8.200	15.300	22.220
--------	--------	-------	--------	--------

The dip array after calling mult_array():

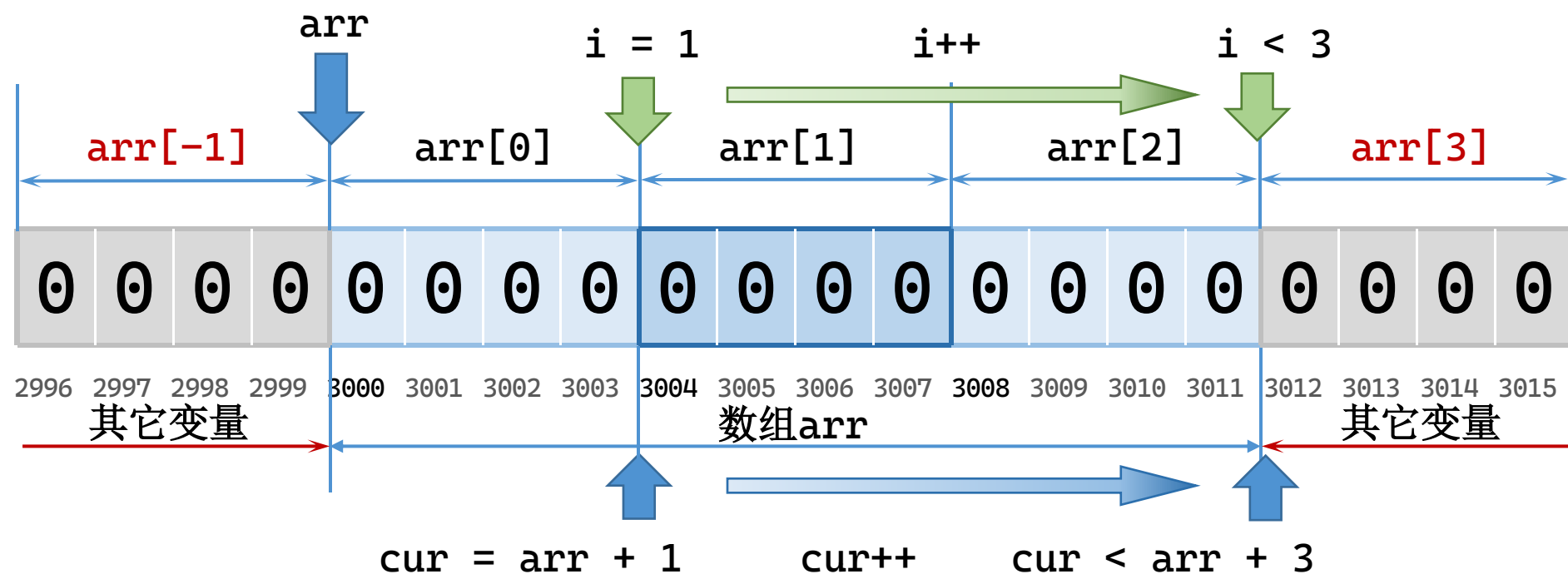
50.000	44.150	20.500	38.250	55.550
--------	--------	--------	--------	--------

函数在参量使用指针操作数组

- 函数的参量传递

- 数组名 a 和长度 N

- 起始位置 s 和结束位置 e



```

/* sum_arr2.c -- sums the elements of an array */
#include <stdio.h>
#define SIZE 10
int sump(int* start, int* end);
int main(void) {
    int marbles[SIZE] = { 20,10,5,39,4,16,19,26,31,20 };
    long answer;
    answer = sump(marbles, marbles + SIZE);
    printf("The total number of marbles is %ld.\n", answer);
    return 0;
}

/* use pointer arithmetic */
int sump(int* start, int* end) {
    int total = 0;
    while (start < end) {
        total += *start; // add value to total
        start++;         // advance pointer to next element
    }
    return total;
}

```

The total number of marbles is 190.

函数用多维数组作为参量

- 函数参量中使用多维数组

- 指出除最高维以外的维度，另设参量为最高维限度

函数形式参量声明	说明
<code>int sum2(int ar[][[]], int rows);</code>	错误，低维长度未指定
<code>int sum2(int ar[][4], int rows);</code>	正确
<code>int sum2(int ar[3][4], int rows);</code>	正确，但最高维长度（3）无效

- 函数中使用数组应正确区分指针和数组

函数形式参量声明	参量ar的含义
<code>int sum4d(int ar[][12][20][30], int rows);</code>	四维数组
<code>int sum4d(int (*ar)[12][20][30], int rows);</code>	指向三维数组的指针

```
// array2d.c -- functions for 2d arrays
#include <stdio.h>
#define ROWS 3
#define COLS 4
void sum_rows(int ar[][COLS], int rows);
void sum_cols(int[][COLS], int);    // ok to omit names
int sum2d(int(*ar)[COLS], int rows); // another syntax
int main(void) {
    int junk[ROWS][COLS] = {{2,4,6,8},{3,5,7,9},{12,10,8,6}};
    sum_rows(junk, ROWS);
    sum_cols(junk, ROWS);
    printf("Sum of all elements = %d\n", sum2d(junk, ROWS));
    return 0;
}

void sum_rows(int ar[][COLS], int rows) {
    int r;
    int c;
    int tot;
    for (r = 0; r < rows; r++) {
        tot = 0;
        for (c = 0; c < COLS; c++)
            tot += ar[r][c];
    }
}
```

```

        printf("row %d: sum = %d\n", r, tot);
    }
}

void sum_cols(int ar[][COLS], int rows) {
    int r;
    int c;
    int tot;
    for (c = 0; c < COLS; c++) {
        tot = 0;
        for (r = 0; r < rows; r++)
            tot += ar[r][c];
        printf("col %d: sum = %d\n", c, tot);
    }
}

int sum2d(int ar[][COLS], int rows) {
    int r;
    int c;
    int tot = 0;

    for (r = 0; r < rows; r++)
        for (c = 0; c < COLS; c++)
            tot += ar[r][c];
    return tot;
}

```

```

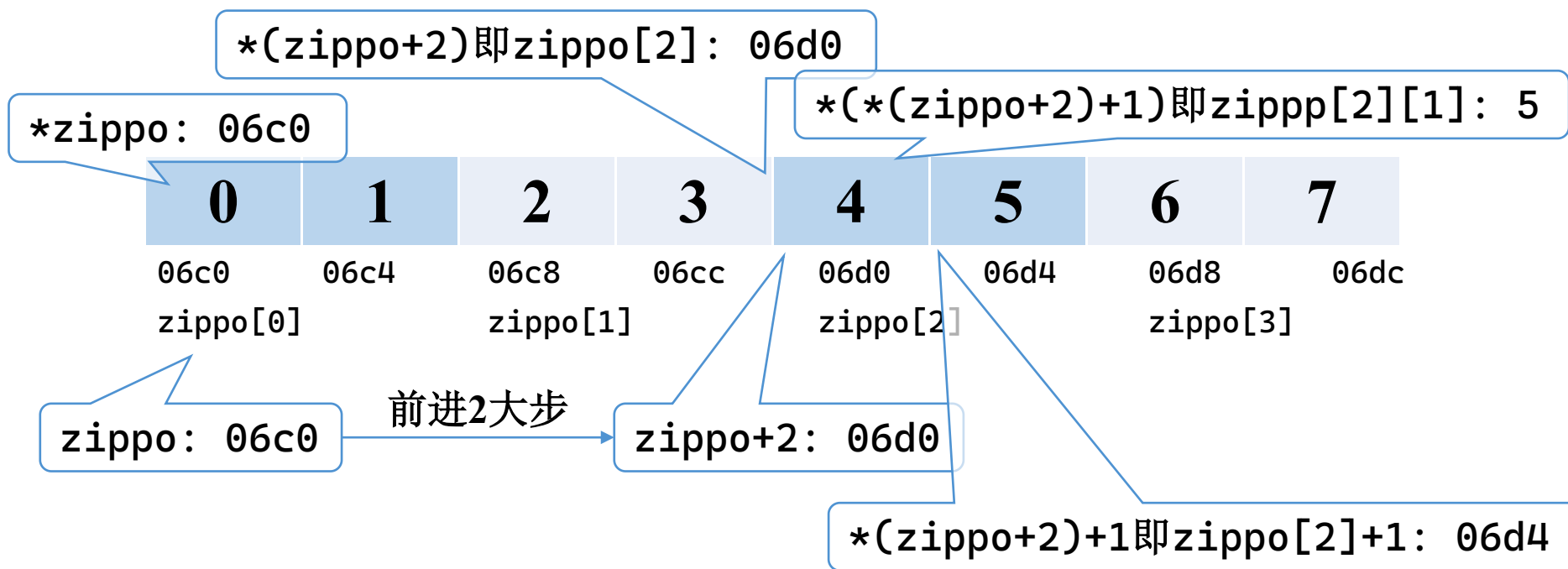
row 0: sum = 20
row 1: sum = 24
row 2: sum = 36
col 0: sum = 17
col 1: sum = 19
col 2: sum = 21
col 3: sum = 23
Sum of all elements = 80

```

多维数组下标及其指针形式

- 多维数组下标层次与间接寻址层次顺序相同

```
int zippo[4][2]; /* an array of arrays of ints */
```



```

/* zippo1.c -- zippo info */
#include <stdio.h>
int main(void) {
    int zippo[4][2] = { { 2, 4 }, { 6, 8 }, { 1, 3 }, { 5, 7 } };

    printf("    zippo = %p,    zippo + 1 = %p\n", zippo, zippo + 1);
    printf("zippo[0] = %p, zippo[0] + 1 = %p\n", zippo[0], zippo[0] + 1);
    printf("    *zippo = %p,    *zippo + 1 = %p\n", *zippo, *zippo +
1);
    printf("zippo[0][0] = %d\n", zippo[0][0]);
    printf("    *zippo[0] = %d\n", *zippo[0]);
    printf("        **zippo = %d\n", **zippo);
    printf("            zippo[2][1] = %d\n", zippo[2][1]);
    printf("*(*(zippo+2) + 1) = %d\n", *(*(zippo+2) + 1));

    return 0;
}

```

```

zippo = 009DF850,    zippo + 1 = 009DF858
zippo[0] = 009DF850, zippo[0] + 1 = 009DF854
*zippo = 009DF850,    *zippo + 1 = 009DF854
zippo[0][0] = 2
*zippo[0] = 2
**zippo = 2
zippo[2][1] = 3
*(*(zippo+2) + 1) = 3

```


指向数组的指针

- 定义指向多维数组的指针 `zippo`的类型是“指向 `int[2]` 的指针”

```
int zippo[4][2];      /* an array of arrays of ints */
```

- 可以被`zippo`赋值的`p_zippo`定义

```
int (*p_zippo)[2];      /* a pointer to the array zippo */
```

- 不可以被`zippo`赋值的`p_zippo`定义

```
int *p_zippo;      /* an improper pointer to the array zippo */  
int **p_zippo;      /* an improper pointer to the array zippo */  
int (*p_zippo)[4];      /* a wrong pointer to the array zippo */  
int *p_zippo[2];      /* a wrong pointer to the array zippo */
```

指向单个 `int` 的指针

指向 `int[4]` 的指针

指向“`int *`”的指针（指针的指针）

含 2 个 `int *` 的数组

```

/* zippo2.c -- zippo info via a pointer variable */
#include <stdio.h>
int main(void) {
    int zippo[4][2] = { { 2, 4 }, { 6, 8 }, { 1, 3 }, { 5, 7 } };
    int (*pz)[2];
    pz = zippo;
    printf("    pz = %p,    pz + 1 = %p\n", pz, pz + 1);
    printf("pz[0] = %p, pz[0] + 1 = %p\n", pz[0], pz[0] + 1);
    printf("    *pz = %p,    *pz + 1 = %p\n", *pz, *pz + 1);
    printf("pz[0][0] = %d\n", pz[0][0]);
    printf("    *pz[0] = %d\n", *pz[0]);
    printf("        **pz = %d\n", **pz);
    printf("            pz[2][1] = 3\n");
    printf("*(*(pz+2) + 1) = 3\n");
    return 0;
}

```

```

pz = 00B0FEE8,    pz + 1 = 00B0FEF0
pz[0] = 00B0FEE8, pz[0] + 1 = 00B0FEEC
    *pz = 00B0FEE8,    *pz + 1 = 00B0FEEC
pz[0][0] = 2
    *pz[0] = 2
        **pz = 2
            pz[2][1] = 3
*(*(pz+2) + 1) = 3

```

目录

	3	指针
	3.3	指针和数组
	3.4	指针的应用
	3.5	常见的编程错误
	4	变长数组和复合文字

利用指针改变函数中变量的值

- 普通参量传递时，无法修改值
 - 变量名只能在声明语句所在的代码块中使用
 - 参量是按值传递，参量值改变，不影响参数值的改变
- 永久改变参量的值，应利用指针
 - 在生命周期内，作用域外修改变量值
 - 参量传入内存地址，函数内操作内存地址，永久修改

```

/* swap1.c -- first attempt at a swapping function */
#include <stdio.h>
void interchange(int u, int v); /* declare function */
int main(void)
{
    int x = 5, y = 10;
    printf("Originally x = %d and y = %d.\n", x , y);
    interchange(x, y);
    printf("Now x = %d and y = %d.\n", x, y);
    return 0;
}
void interchange(int u, int v) /* define function */
{
    int temp;
    temp = u;
    u = v;
    v = temp;
}

```

Originally x = 5 and y = 10.
Now x = 5 and y = 10.

```

/* swap2.c -- researching swap1.c */
#include <stdio.h>
void interchange(int u, int v);
int main(void) {
    int x = 5, y = 10;
    printf("Originally x = %d and y = %d.\n", x , y);
    interchange(x, y);
    printf("Now x = %d and y = %d.\n", x, y);
    return 0;
}
void interchange(int u, int v)
    int temp;
    printf("Originally u = %d and v = %d.\n", u , v);
    temp = u;
    u = v;
    v = temp;
    printf("Now u = %d and v = %d.\n", u, v);
}

```

Originally x = 5 and y = 10.
 Originally u = 5 and v = 10.
 Now u = 10 and v = 5.
 Now x = 5 and y = 10.

利用函数交换两个变量的值

- 左侧代码无法交换，右侧代码可以交换

```
#include <stdio.h>
int swap(int a, int b)
{
    int t;
    t = a; a = b; b = t;
    printf("%d %d\n", a, b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(a, b);
    printf("%d %d\n", a, b);
    return 0;
}
```

```
#include <stdio.h>
int swap(int *a, int *b)
{
    int t;
    t = *a; *a = *b; *b = t;
    printf("%d %d\n", *a, *b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

利用函数交换两个变量的值

- 左侧代码无法交换，右侧代码可以交换

```
#include <stdio.h>
int swap(int a, int b)
{
    int t;
    t = a; a = b; b = t;
    printf("%d %d\n", a, b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(a, b);
    printf("%d %d\n", a, b);
    return 0;
}
```

环节（左侧）	进入 swap前	刚进入 swap	退出 swap前	退出 swap后
main中a值	3	3	3	3
main中a地址	0x3008	0x3008	0x3008	0x3008
main中b值	4	4	4	4
main中b地址	0x3004	0x3004	0x3004	0x3004
swap中a值	/	3	4	/
swap中a地址	未开辟	0x4008	0x4008	已释放
swap中b值	/	4	3	/
swap中b地址	未开辟	0x4004	0x4004	已释放

利用函数交换两个变量的值

- 左侧代码无法交换，右侧代码可以交换

环节（右侧）	进入 swap前	刚进入 swap	退出 swap前	退出 swap后
main中a值	3	3	4	4
main中a地址	0x3008	0x3008	0x3008	0x3008
main中b值	4	4	3	3
main中b地址	0x3004	0x3004	0x3004	0x3004
swap中a值	/	0x3008	0x3008	/
swap中a地址	未开辟	0x4008	0x4008	已释放
swap中b值	/	0x3004	0x3004	/
swap中b地址	未开辟	0x4004	0x4004	已释放

```
#include <stdio.h>
int swap(int *a, int *b)
{
    int t;
    t = *a; *a = *b; *b = t;
    printf("%d %d\n", *a, *b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

利用函数交换两个变量的值

- 左侧错误：函数内外，即便同名变量也是不同的函数

环节 (左侧)	main中 a值	main中 a地址	main中 b值	main中 b地址	swap中 a值	swap中 a地址	swap中 b值	swap中 b地址
进入swap之前	3	0x3008	4	0x3004	/	未开辟	/	未开辟
刚进入swap	3	0x3008	4	0x3004	3	0x4008	4	0x4004
退出swap之前	3	0x3008	4	0x3004	4	0x4008	3	0x4004
退出swap之后	3	0x3008	4	0x3004	/	已释放	/	已释放

环节 (右侧)	main中 a值	main中 a地址	main中 b值	main中 b地址	swap中 a值	swap中 a地址	swap中 b值	swap中 b地址
进入swap之前	3	0x3008	4	0x3004	/	未开辟	/	未开辟
刚进入swap	3	0x3008	4	0x3004	0x3008	0x4008	0x3004	0x4004
退出swap之前	4	0x3008	3	0x3004	0x3008	0x4008	0x3004	0x4004
退出swap之后	4	0x3008	3	0x3004	/	已释放	/	已释放

将一组数据存储到文件

- 不同类型指针的相互转换

- 常见于和字节数组互转 (`unsigned char *`)

- 常见于开辟空间时 `p = (int *)malloc(sizeof(int) * 60);`

在实际写程序时，经常需要把“某种类型的数据”当成一串原始字节来处理（比如写入文件、发到网络、做加密/压缩等），这时就会在某种类型的指针）和 `unsigned char *`（字节数组指针）之间来回转换，也就是“互转”。

```
struct Point {  
    int x;  
    int y;  
};  
struct Point p = {10, 20};  
// 想把这个结构体按字节写到文件里  
unsigned char *buf = (unsigned char *)&p; //这里就是和 unsigned char* 的互转  
fwrite(buf, 1, sizeof(p), fp);
```

小结：指针的应用场景

- 在生命周期之内、作用域之外访问变量
 - 在作用域内使用变量名访问
 - 在作用域外无法通过变量名访问，应使用指针
- 在栈外开辟和使用较大的内存空间
 - 开辟后的内存空间应赋值给指针后使用

目录

	3	指针
	3.4	指针的应用
	3.5	常见的编程错误
	4	变长数组和复合文字
	5	排序算法

空指针解引用

- 空指针解引用，将产生运行错误

```
int foo(int* p) {  
    int x = *p;  
    if (!p)  
        return x;  
    else  
        return 0;  
}  
  
int bar() {  
    int* p = NULL;  
    return *p;  
}
```

程序会试图解引用一个空指针，这会导致运行时错误或程序崩溃。

非法标量

- 错误的强制类型转换，将产生运行错误

```
int f(void) {  
    _Bool b = 0;  
    unsigned char* p = (unsigned char*)&b;  
    *p = 10;  
    return b == 0;  
}
```

直接将内存中的值改为非 0 或 1（如 10）会违反 `_Bool` 的规则。

目录

	1	内存的组织
	2	数组
	3	指针
	4	变长数组和复合文字
	5	排序算法

变长数组（C99标准）

- C99允许声明数组时，使用变量指示长度
- 变长数组是声明时用变量表示长度，并非长度可变

```
int quarters = 4;  
int regions = 5;  
double sales[regions][quarters]; // a VLA
```

- 在函数参量中使用变长数组

```
int sum2d(int rows, int cols, int ar[rows][cols]); // ar a VLA
```

```
int sum2d(int ar[rows][cols], int rows, int cols); // invalid order
```

```
int sum2d(int, int, int ar[*][*]); // ar a VLA, names omitted
```

变长数组（C99标准）

- 不推荐使用变长数组

- 变长数组本质上是动态分配的数组，建议改用：

- 开辟空间：指针=malloc(字节数)；释放空间：free(指针)

- 变长数组限制

- 由于编译时长度未知，**声明时不能初始化**

- 必须在程序块的范围定义，**不能在文件范围内定义**

- 作用域和生存时间为块的范围，**不能是静态的或者外部的**

- **不能作为结构体或联合体的成员**，只能为独立数组形式

`int a[n] = {1, 2, 3}; // 不允许` `static int a[n]; // 不允许`

`int n; // 全局变量`
`int a[n]; // 不允许`

`int n;`
`struct S {`
`int a[n]; // 不允许`
`};`

```

//vararr2d.c -- functions using VLAs
#include <stdio.h>
#define ROWS 3
#define COLS 4
int sum2d(int rows, int cols, int ar[rows][cols]);

int main(void) {
    int i, j;
    int rs = 3;
    int cs = 10;
    int junk[ROWS][COLS] = {{2,4,6,8},{3,5,7,9},{12,10,8,6}};
    int morejunk[ROWS - 1][COLS + 2] = {{20,30,40,50,60,70},
        {5,6,7,8,9,10}};
    int varr[rs][cs]; // VLA
    for (i = 0; i < rs; i++)
        for (j = 0; j < cs; j++)
            varr[i][j] = i * j + j;
    printf("3x5 array\n");
    printf("Sum of all elements = %d\n",
        sum2d(ROWS, COLS, junk));
}

```

```

printf("2x6 array\n");
printf("Sum of all elements = %d\n",
      sum2d(ROWS - 1, COLS + 2, morejunk));
printf("3x10 VLA\n");
printf("Sum of all elements = %d\n",
      sum2d(rs, cs, varr));
return 0;
}

// function with a VLA parameter
int sum2d(int rows, int cols, int ar[rows][cols]) {
    int r;
    int c;
    int tot = 0;
    for (r = 0; r < rows; r++)
        for (c = 0; c < cols; c++)
            tot += ar[r][c];
    return tot;
}

```

复合文字

- 普通数组的声明方法

```
int diva[2] = {10, 20};
```

- 复合文字

- 没有名称，只能通过赋值给指针（数组不可以）使用

```
(int [2]){10, 20} // a compound literal           “长度未显式给出”的数组类型
(int []){50, 20, 90} // a compound literal with 3 elements
int sum(const int ar[], int n);
...
int total3;
total3 = sum((int []){4,4,4,5,5,5}, 6); // 用符合文字给参数传参
int (*pt2)[4]; // declare a pointer to an array of 4-int arrays
pt2 = (int [2][4]) { {1,2,3,-9}, {4,5,6,-8} }; //多维数组符合文字
```

```
// flc.c -- funny-looking constants
#include <stdio.h>
#define COLS 4
int sum2d(const int ar[][COLS], int rows);
int sum(const int ar[], int n);
int main(void) {
    int total1, total2, total3;
    int* pt1;
    int(*pt2)[COLS];
    pt1 = (int[2]){ 10, 20 };
    pt2 = (int[2][COLS]){ {1,2,3,-9}, {4,5,6,-8} };
    total1 = sum(pt1, 2);
    total2 = sum2d(pt2, 2);
    total3 = sum((int[]) { 4, 4, 4, 5, 5, 5 }, 6);
    printf("total1 = %d\n", total1);
    printf("total2 = %d\n", total2);
    printf("total3 = %d\n", total3);
    return 0;
}
```

```
int sum(const int ar[], int n) {
    int i;
    int total = 0;
    for (i = 0; i < n; i++)
        total += ar[i];
    return total;
}

int sum2d(const int ar[][COLS], int rows) {
    int r;
    int c;
    int tot = 0;
    for (r = 0; r < rows; r++)
        for (c = 0; c < COLS; c++)
            tot += ar[r][c];
    return tot;
}
```

目录

	1	指针
	2	变长数组和复合文字
	3	排序算法

冒泡排序法 (Bubble Sort)

- 两层循环

- 内层循环逐个比较，将最大值（泡泡）浮至顶部
- 外层循环控制内层循环的范围，已经浮出的元素不需再比较

size	i	a[0]	a[1]	a[2]	a[3]
4	1	4	3	2	1
4	2	3	4	2	1
4	3	3	2	4	1
3	1	3	2	1	4
3	2	2	3	1	4
2	1	2	1	3	4
-	-	1	2	3	4

```
for (size = len; size > 1; size--)  
    for (i = 1; i < size; i++)  
        swap(arr[i - 1], arr[i]);
```

size	i	a[0]	a[1]	a[2]	a[3]
3	1	3	2	1	4

Diagram illustrating the range of the inner loop. A horizontal arrow labeled "size" spans from the column index *i* (which is 1) to the end of the array (index 3). The element at index 4 (value 4) is shown in a green box, indicating it is sorted and excluded from the current pass.

```

void bubble_sort_1(double arr[], const unsigned int len)
{
    unsigned int size, i;
    for (size = len; size > 1; size--) // 排序的范围从全数组
慢慢收缩至停止
    {
        for (i = 1; i < size; i++) // 单趟排序
        {
            if (arr[i - 1] > arr[i]) // 需要发生交换的条件
            {
                double temp; // temp的唯一正确用法
                temp = arr[i - 1];
                arr[i - 1] = arr[i];
                arr[i] = temp;
            }
        }
    }
}

```

```

void bubble_sort_2(double arr[], const unsigned int len)
{
    unsigned int sorted, i; // sorted是排好序的下标, i是索引
    for (sorted = 0; sorted < len; sorted++) // 排序的范围
        从全数组慢慢收缩至停止, 虽然此处是递增的, 但是在下面使用时有减号
    {
        for (i = 1; i < len - sorted; i++) // 单趟排序
        {
            if (arr[i - 1] > arr[i]) // 需要发生交换的条件
            {
                double temp; // temp的唯一正确用法
                temp = arr[i - 1];
                arr[i - 1] = arr[i];
                arr[i] = temp;
            }
        }
    }
}

```

冒泡排序法（加强版）

- 增加一个标记量
 - 如果某次内层排序没有实际发生排序，则认定为已经排好序，循环终止。

size	i	a[0]	a[1]	a[2]	a[3]
4	1	4	1	2	3
4	2	1	4	2	3
4	3	1	2	4	3
3	1	1	2	3	4
3	2	1	2	3	4
2	1	1	2	3	4
-	-	1	2	3	4

```

void bubble_sort_3(double arr[], const unsigned int len)
{
    unsigned int size, i; // size是排序的范围, i是索引
    for (size = len; size > 1; size--) // 排序范围从全数组收缩至停止
    {
        int is_swapped = 0; // 标记是否交换
        for (i = 1; i < size; i++) // 单趟排序
        {
            if (arr[i - 1] > arr[i]) // 需要发生交换的条件
            {
                double temp; // temp的唯一正确用法
                temp = arr[i - 1];
                arr[i - 1] = arr[i];
                arr[i] = temp;
                is_swapped = 1; // 标记已经经过交换
            }
        }
        if (!is_swapped) // 如果未经交换, 说明已排好序, 可退出不必再排
            break;
    }
}

```

选择排序法 (Select Sort)

- 两层循环

- 内层循环逐个比较，将最大值与所应在位置置换
- 外层循环控制内层循环的范围，已经安排的位置不再比较

pos	min_i	a[0]	a[1]	a[2]	a[3]
0	2	4	3	5	1
1	2	5	3	4	1
2	2	5	4	3	1
3	-	5	4	3	1

pos	j	a[0]	a[1]	a[2]	a[3]
0	1	4	3	5	1

pos+1 len

```
for (pos = 0; pos < len - 1; pos++)  
    for (j = pos + 1; j < len; j++)
```

```
void select_sort(double arr[], const unsigned int len) {  
    for (unsigned int i = 0; i < len - 1; i++) {  
        int min_i = i;  
        for (unsigned int j = i + 1; j < len; j++)  
            if (arr[j] < arr[min_i])  
                min_i = j;  
        if (min_i != i) {  
            double temp = arr[i];  
            arr[i] = arr[min_i];  
            arr[min_i] = temp;  
        }  
    }  
}
```

快速排序法 (Quick Sort)

- 选择元素，将小放到左边，大放到右边，再分别排序

low	hig	i	j	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]
0	8	1	5	4	7	2	9	3	1	5	6	8
				x	从左向右找出大于x的数				从右向左找出小于x的数			
		3	4	1	*	2	9	3	7	5	6	8
		3	3	1	3	2	*	9	7	5	6	8
lo	hi	lo	hi	1	3	2	4	9	7	5	6	8
0	0	8	8	1	3	2	4	8	7	5	6	9
2	2	7	7	1	2	3	4	6	7	5	8	9
		5	6	1	2	3	4	5	6	7	8	9
		5	5	1	2	3	4	5	6	7	8	9


```

void quick_sort(double arr[], const int left, const int right)
{
    if (left < right) {
        double key = arr[left];
        int low = left;
        int high = right;
        while (low < high) {
            while (low < high && arr[high] > key)
                high--;
            arr[low] = arr[high];
            while (low < high && arr[low] < key)
                low++;
            arr[high] = arr[low];
        }
        arr[low] = key;
        quick_sort(arr, left, low - 1);
        quick_sort(arr, low + 1, right);
    }
}

```

例题：字符串排序

- 在字符串排序时，尽量使用指针而不是字符串复制
 - 字符串的复制比指针赋值耗费的时间更长
 - 将指针或下标存入数组也需要耗费时间

字符串复制

```
for (i=0; i<STR_COUNT; i++) {  
    for (j=0; j<STR_COUNT-1-i; j++)  
        if(strcmp(str[j],str[j+1])>0) {  
            strcpy(s,str[j]);  
            strcpy(str[j],str[j+1]);  
            strcpy(str[j+1],s);  
        }  
}
```

指针/下标复制

```
for (i=0; i<STR_COUNT; i++)  
    for (j=0; j<STR_COUNT-1-i; j++)  
        if (strcmp(str[index[j]],  
str[index[j+1]])>0) {  
            temp=index[j];  
            index[j]=index[j+1];  
            index[j+1]=temp;  
        }
```

```

/* sort_str.c -- reads in strings and sorts them */
#include <stdio.h>
#include <string.h>
#define SIZE 81          /* string length limit, including \0 */
#define LIM 20           /* maximum number of lines to be read */
#define HALT ""          /* null string to stop input */

void stsort(char* strings[], int num); /* string-sort function */
char* s_gets(char* st, int n);

int main(void)
{
    char input[LIM][SIZE]; /* array to store input */
    char *ptstr[LIM];      /* array of pointer variables */
    int ct = 0;            /* input count */
    int k;                 /* output count */

    printf("Input up to %d lines, and I will sort them.\n", LIM);
    printf("To stop, press the Enter key at a line's start.\n");

```

```

while (ct < LIM && s_gets(input[ct], SIZE) != NULL
      && input[ct][0] != '\0')
{
    ptstr[ct] = input[ct]; /* set ptrs to strings */
    ct++;
}
stsrst(ptstr, ct); /* string sorter */
puts("\nHere's the sorted list:\n");
for (k = 0; k < ct; k++)
    puts(ptstr[k]); /* sorted pointers */
return 0;
}

/* string-pointer-sorting function */
void stsrst(char *strings[], int num)
{
    char *temp;
    int top, seek;

```

```

for (top = 0; top < num-1; top++)
    for (seek = top + 1; seek < num; seek++)
        if (strcmp(strings[top], strings[seek]) > 0)
        {
            temp = strings[top];
            strings[top] = strings[seek];
            strings[seek] = temp;
        }
}

char * s_gets(char * st, int n)
{
    char * ret_val;
    int i = 0;
    ret_val = fgets(st, n, stdin);
    if (ret_val)
    {
        while (st[i] != '\n' && st[i] != '\0')

```

```

        i++;
    if (st[i] == '\n')
        st[i] = '\0';
    else // must have words[i] == '\0'
        while (getchar() != '\n')
            continue;
}
return ret_val;
}

```

Input up to 20 lines, and I will sort them.
To stop, press the Enter key at a line's start.

notice↵
Be careful↵
note 4↵
!!↵
~~↵
↵

Here's the sorted list:

!!
 Be careful
 note 4
 notice
 ~~

谢谢观看



福州大学
FUZHOU UNIVERSITY